

Project Proposal: An Agentic AI System for Long-Term Friendship Maintenance

1. Problem Statement & Motivation

Real-World Business Problem

Maintaining long-term personal relationships is a persistent challenge in modern life. As individuals balance academic, professional, and personal responsibilities, meaningful friendships often weaken, not due to lack of care, but due to forgetfulness, cognitive overload, and the absence of structured support. While people intend to stay in touch, they frequently miss important moments such as birthdays, life transitions, or periods when friends may need emotional support.

Existing tools such as calendars, contact lists, and social media platforms do not effectively support long-term relationship maintenance. These tools lack contextual memory, emotional awareness, and proactive assistance. As a result, users rely entirely on their own memory and initiative, leading to unintentional relationship decay over time. This represents a real-world productivity and well-being problem that impacts personal satisfaction and social connectedness. Unlike traditional reminder apps or generic calendar alerts, this system aims to integrate contextual memory (what is happening in a friend's life), relationship intent (why the user cares), and timing decisions (when a prompt is genuinely helpful) into a unified, adaptive assistant.

Target Users

The primary target users for this system include:

- College students and young professionals with busy schedules
- Individuals managing large or geographically distributed social networks
- Users who value intentional, long-term relationships but struggle with consistency

These users are not seeking automation of relationships, but rather support that helps them show up more thoughtfully and consistently.

Why an Agentic AI Solution Is Appropriate

An agentic AI solution is well-suited to this problem because maintaining relationships is not a static or rule-based task. The workflow cannot be fully predefined by a human or external system due to the following reasons:

- Relationship needs change over time
- The importance and urgency of interactions vary by context
- Decisions must be made autonomously about when and how to prompt the user

An agentic system can observe interaction patterns, maintain long-term memory, plan future actions, and adapt reminders dynamically. Unlike a simple reminder app, an agentic AI can reason about context, prioritize users' relationships, and act proactively while still keeping the human user in control.

2. Use Cases & Scenarios

Scenario 1: Maintaining Long-Distance Friendships

Use Case:

A user wants to stay connected with a close friend who lives in another city and whom they do not speak to regularly.

Agent Tasks:

- Track the date and frequency of past interactions
- Tracks timezones and finds a suitable time
- Detect prolonged inactivity
- Determine an appropriate time to prompt the user
- Suggest a low-effort check-in message or activity idea
- Update memory based on user response or dismissal

Scenario 2: Supporting Friends During Important Life Events

Use Case:

A user wants to be more intentional about checking in on a friend who is experiencing a stressful life event (e.g., exams, job search, relocation).

Agent Tasks:

- Store contextual notes provided by the user
- Schedule future follow-up reminders
- Adjust reminder tone and frequency based on context
- Generates ideas for messages while keeping it fully focused on the user
- Avoid excessive or inappropriate nudging

Scenario 3: Reconnecting with Dormant Relationships

Use Case:

A user wants to reconnect with friends they have unintentionally lost touch with over time.

Agent Tasks:

- Identify long periods of inactivity
- Rank reconnection opportunities by closeness and history
- Suggest low-pressure reconnection prompts
- Track outcomes and update relationship status

Success in this scenario is measured by the user maintaining a steady cadence of contact (e.g., at least monthly) with long-distance friends they identify as important.

3. Initial System Design

Preliminary List of Agents and Roles

1. Orchestrator Agent

- Routes tasks and information between the Memory, Planner, Suggestion Generator, and Critic / Safety Agents.
- Manages the overall workflow: when the user opens the app or a scheduled check occurs, it decides which agents to invoke, in what order, and with what context.
- Ensures that each suggestion is generated from up-to-date memory, checked by the Critic, and then surfaced to the user for final approval.

2. Memory Agent

- Stores friend profiles, interaction history, and contextual notes
- Maintains long-term memory across sessions

3. Planner Agent

- Determines which relationships require attention
- Decides when reminders should occur based on patterns and context

4. Suggestion Generator Agent

- Produces message drafts, conversation starters, or activity ideas
- Adapts tone and content to the relationship type

5. Critic / Safety Agent

- Evaluates suggestions for appropriateness and relevance
- Prevents over-notification or socially awkward prompts

These agents operate in a coordinated workflow, allowing the system to reason, plan, and act autonomously while remaining user-centered. To protect privacy, these agents will operate on pseudonymized profiles and high-level context (e.g., 'close friend, stressed about exams') rather than raw identifiers or full message histories.

Expected Tools, APIs, and Data Sources

- Large Language Model (LLM) API for reasoning and generation
- Local storage (e.g., SQLite or JSON files) for persistent memory and privacy

- Backend framework (e.g., Python with FastAPI)
- Web-based front-end interface for user interaction
- Optional notification or calendar APIs for reminders

4. Feasibility & Risks

Potential Challenges

- Hallucination: Incorrect recall of stored context. Mitigation: constrain the system so that suggestions are generated only from the explicit memory records (dates, tags, short summaries) and are always shown to the user for approval and editing before sending.
- Context drift: Loss of relevance over extended time periods
- Over-notification: Risk of user fatigue. Mitigation: enforce hard caps on daily prompts, allow users to easily snooze or mute contacts, and start with conservative defaults for reminder frequency.
- Tool failures: API downtime or rate limits
- Privacy limitations: The system will only store user-provided information, and identifiers will be stored separately from higher-level summaries. All reminders will be computed on local or user-controlled storage, and users can delete data or disable memory at any time.

Constraints

- Privacy limitations: The system only stores user-provided information
- Rate limits on LLM API usage
- Computational constraints for running multiple agents
- Ethical constraints around autonomy and user consent
- Depends on the user to give information for specific contacts

These risks will be mitigated through transparent memory handling, conservative reminder strategies, and explicit user control over data and notifications.

5. Timeline for Project Execution

The project will be completed within the class duration:

- Weeks 1- 2: Proposal finalization and system design
- Weeks 3 - 4: Front-end interface development (Milestone I)
- Weeks 5 - 6: Multi-agent workflow implementation (Milestone II)
- Weeks 7 - 8: Evaluation plan and pilot testing (Milestone III)
- Final Weeks: Refinement, final implementation, report, and presentation